Kipp: Oracle-Gated, Placement-Invariant Paged Attention for Verifiable LLM Inference

David Khachatryan Johansson
Cortify
Stockholm, Sweden
david@cortify.io

ABSTRACT

Paged key–value (KV) caches are standard in LLM serving, and every system that relocates cache blocks at runtime—for prefix sharing, migration, spilling, or speculative rollback—relies on an assumption none of them tests: the physical placement of KV blocks must not change the model's output. We turn that assumption into a machine-checked contract. Placement invariance is stated as a proposition whose hypotheses map to real bug classes and enforced by a *scramble gate* that reverses a sequence's block table and requires bitwise-identical logits. A mutation study shows why tolerance-based reference testing is not enough: of four seeded, realistic KV-addressing faults, a block-rollover bug measures NMSE of exactly 0 against the reference—invisible to any tolerance, however tight—and only the scramble gate catches it, while two read-side faults never flip the arg max and fall only to a tight NMSE bound, and a fourth is gross corruption any check catches. The two gate families are complementary, and the study also repaired a blind spot in our own gate. We embody the discipline in Kipp, a hand-written C11 engine for the Qwen3 dense family whose byte-stable scalar oracle gates two GPU backends and every feature: 8- and 4-bit quantization, simdgroup-matrix prefill, continuous batching, adaptive speculation, and pooled KV sharing. Kipp wins bandwidth-bound decode against llama.cpp on identical hardware, prefix reuse cuts time to first token by 175×, and every benchmark number in this paper is bound to a committed, re-runnable artifact.

1 INTRODUCTION

Serving a large language model (LLM) is dominated by the cost of the key-value (KV) cache. Kwon et al. [22] observed that reserving a contiguous KV region per request wastes 60–80% of KV memory to internal and external fragmentation, and introduced *PagedAttention*: store KV in fixed-size blocks and give each sequence a *block table* that maps logical positions to arbitrary physical blocks, exactly as an operating system pages virtual memory. Paging is now standard practice [22, 32, 47], and it underpins prefix sharing, live block migration, and speculative-decode rollback.

Paging rests on an unstated assumption: where a block physically lives must not change what the model computes. Systems that relocate KV at runtime take this for granted, whether they migrate running requests between instances [42], spill cache entries between GPU and host memory [23], ship KV between prefill and decode clusters [36], persist it across conversation turns [12], or share it across calls [47]. Of the systems Table 1 surveys, ten relocate KV through an indirection and none tests placement invariance (the eleventh, vAttention, sidesteps the indirection by construction). However, the indirection they all depend on, a per-access `block_table[pos>>5]*32 + (pos&31)` lookup replicated independently in every backend's write and read kernels, is exactly where off-by-one and stride bugs hide. Such bugs are silent. They corrupt a few positions without crashing, and a tolerance-based comparison to a reference forward pass, which is the standard test, can mask them as numerical noise (Section 3).

Kipp takes the opposite stance: correctness properties that serving systems rely on should be adversarially tested, continuously, on every backend. This paper makes the following contributions.

(1) We state placement invariance as a proposition with explicit hypotheses, each of which corresponds to a real bug class, and realize it as a *scramble gate*: reverse a sequence's block table and require the output logits to be bitwise identical to the identity mapping (Section 4).
(2) We evaluate paging-test adequacy by mutation. Seeding four realistic, asymmetric KV-addressing bugs shows that the detection pattern splits exactly along the proposition's hypotheses: placement-commuting faults are caught only by reference testing, two of them without ever flipping the arg max, and the placement-coupled rollover fault, which measures NMSE of exactly 0 against the reference, is caught only by the scramble gate. The study also exposed a blind spot in our own gate and drove its fix (Section 7.3).
(3) We hold a scalar CPU forward pass byte-identical across the engine's evolution and use it as a hard gate: Metal and CUDA must match its arg max exactly and stay within 10^{-4} NMSE (Section 5).
(4) We build and evaluate the engine itself (Table 3): a C11 core running the Qwen3 dense family with 8- and 4-bit quantization, simdgroup-matrix prefill, continuous batching, an OpenAI-compatible server, and prompt-lookup speculation with adaptive gating, every feature admitted through the same gates, and evaluated end to end, from microbenchmarks and quantization quality to serving under load and a same-hardware llama.cpp comparison (Sections 6 and 7).
(5) We bind every benchmark number in this paper to a committed result file produced by the repository's harness, and every correctness claim to a gate command listed in the artifact appendix; the two out-of-band measurements are marked as such where they appear.

We are explicit about what is not new. Attention is invariant to a permutation of stored KV under a matching permutation of the causal mask; this is folklore and has been treated formally in recent KV-quantization work. Zero-copy speculative rollback is standard

paged-cache behavior [24]. Our contribution is the engineering discipline of testing the assumption, the measurement of what existing test regimes miss, and the artifact that results.

2 BACKGROUND

2.1 Transformer inference and the KV cache

Autoregressive inference has two phases with opposite bottlenecks. *Prefill* evaluates the whole prompt at once and is compute-bound, since every weight is reused across all prompt tokens. *Decode* produces one token per step and is bandwidth-bound: each step streams every weight from memory to produce a single token, so throughput tracks bytes per weight. This is why weight quantization accelerates decode but not prefill (Section 7.4). Each generated token appends a key and value vector per layer to the KV cache, which grows linearly with context and persists across steps.

Kipp targets a fixed registry of pinned Qwen3 dense checkpoints (0.6B–32B, base and instruct) [44]. The family shares head dimension 128, eight KV heads [2], a vocabulary of 151,936, RMSNorm $\epsilon = 10^{-6}$, NEOX rotary embeddings [41], grouped-query attention, and SwiGLU MLPs; only depth, width, query-head count, and rope θ vary. Fixing the family, rather than accepting arbitrary GGUF files, is what makes exact per-checkpoint validation tractable.

2.2 Paged KV caches and the OS analogy

Kipp stores a sequence's KV in physical blocks of B=32 positions. A per-sequence *block table* π maps logical block b to a physical block $\pi(b)$; position p resolves to physical slot $\pi(\lfloor p/B \rfloor) \cdot B + (p \bmod B)$. The analogy to OS paging is exact: blocks are pages, the block table is a page table, and logical positions are virtual addresses. The identity table $\pi(b)$=b reproduces a contiguous cache byte for byte. Prefix sharing, block migration, and rollback are all realized by editing π, never by moving KV bytes. The analogy also suggests the missing test. An OS would not ship an MMU whose translation path had only ever been exercised with identity mappings, yet that is how paged LLM engines are tested today.

2.3 Speculation on bandwidth-bound hardware

Because decode streams every weight per step, a k+1-token speculative verify forward is nearly free on compute-bound hardware but costs close to k+1 decode steps when weight streaming dominates [24, 38]. Speculation is therefore a net loss at low acceptance, which motivates the adaptive gate of Section 6.5. The same arithmetic makes batched decode scale: one weight read serves every concurrently decoding sequence [46], which Section 7.7 measures.

3 THE SILENT-CORRUPTION PROBLEM

The KV indirection is two instructions, but several distinct bug classes live in it:

- *Block-index errors*: an off-by-one in the logical-block computation reads a neighboring block's KV.
- *Slot errors*: an off-by-one within the block shifts every source position by one slot.
- *Rollover errors*: the first position after a block boundary is addressed relative to the previous block, one past its end.

Table 1: Systems that relocate or reuse KV at runtime, and whether their test suites check placement invariance as a property. Based on inspection of each system's public repository and test suite as of July 2026; we welcome corrections.

System	KV relocation / reuse	Tests inv.?
vLLM [22]	block-table paging, prefix sharing	no
SGLang [47]	radix-tree prefix sharing	no
TensorRT-LLM [33]	paged KV, block reuse	no
TGI [19]	paged KV	no
llama.cpp [13]	unified KV cells, defrag moves	no
Mooncake [36]	prefill→decode KV transfer	no
InfiniGen [23]	GPU–host KV spilling	no
Llumnix [42]	live request + cache migration	no
CachedAttention [12]	cross-turn KV persistence	no
vAttention [35]	contiguous virtual KV, demand paging	n/a[a]
Kipp (this work)	block-table paging, pooled prefix sharing	**yes**[b]

[a] Avoids table indirection by construction (contiguous virtual addresses).
[b] Bitwise scramble gate on CPU and Metal (Section 4.3).

- *Binding errors*: key and value stores, or their strides, are swapped in one kernel's argument list.

Three properties make these bugs dangerous. First, they are silent. The wrong address is still a valid slot, so no bounds are violated, nothing crashes, and generation continues fluently; attention's softmax dilutes a few corrupted source positions into a plausible output distribution. Second, they are masked by tolerance. The standard test compares final logits to a reference within a numerical tolerance chosen to absorb legitimate reduction-order and quantization noise, and a bug that corrupts a few positions can land within or near that band, indistinguishable from noise. Third, some are invisible under the identity mapping. A rollover bug that writes position $32b$ "one past the end of block b−1" lands on exactly the right byte when blocks are laid out contiguously. Under the identity table every happy-path test passes bitwise, and the bug only corrupts once a scheduler actually relocates a block. Section 7.3 demonstrates each of these behaviors with seeded faults.

The exposure grows with exactly the features that make paging valuable: cross-request prefix sharing backs the same physical block with many sequences, defragmentation compacts live blocks with zero recompute, and speculative rollback drops rejected blocks by unlinking. Each new feature adds code that manipulates π. None of it is exercised by tests that only ever run the identity mapping. Table 1 makes the claim auditable: across ten widely used systems that relocate or reuse KV at runtime, we found no test of placement invariance as a property.

4 PLACEMENT-INVARIANT PAGED ATTENTION

4.1 The invariance property

Proposition 4.1 (Physical-placement invariance). *Fix a sequence of logical positions* $0, \ldots, L-1$ *with token embeddings, rotary*

phases, and causal mask defined on logical positions. Let the KV cache be stored in physical blocks of B slots addressed through a block table, and let π, π' be any two injective maps from logical blocks to physical blocks. Suppose:

- **H1** *the same map is applied to every KV write and every KV read (the address is $\pi(\lfloor p/B \rfloor) \cdot B + (p \bmod B)$, and nothing else in the kernel depends on the physical address);*
- **H2** *rotary embeddings and the causal mask are computed from logical positions only; and*
- **H3** *the attention reduction enumerates source positions in a fixed logical order, with a fixed summation order and the same floating-point operations, independent of π.*

Then for every query position the attention output, and hence the final logits, computed under π and π' are bitwise identical.

Proof sketch. Under H1 the bytes written for logical position p and the bytes read for p coincide regardless of π: read $\circ$ write is the identity on logical positions, so by H2 the sequence of (key, value, phase, mask) tuples entering attention at any query position is equal as an indexed sequence, not merely as a set. By H3 the reduction consumes that sequence in the same order with the same operations, and floating-point evaluation is a deterministic function of operand bits and operation order, so every intermediate value is bit-equal. Induction over layers extends equality to the final logits. □

Each hypothesis is load-bearing, and each corresponds to a real bug class. Violating H1 on one side of the indirection is exactly the fault family of Section 3; a mapping error applied consistently to both sides is a harmless relocation, which is why our mutation study must (and does) seed one-sided faults. H2 fails if any kernel derives a rotary phase or mask bound from a physical address. H3 is the batch-invariance axis of He and Thinking Machines Lab [16]. The proposition does not require determinism across batch sizes, only that placement not perturb the reduction; the two axes are orthogonal, and a kernel can satisfy either without the other.

4.2 Scope, stated precisely

Invariance holds for the physical placement of retained KV, that is, for which physical block holds a given logical position. It does not hold for reordering the logical positions themselves, since rotary embeddings and the causal mask are position-dependent. Nor is the triangular prefill mask invariant to permuting columns of the score matrix. We test and claim only physical-placement invariance. This distinction is what keeps the contract honest.

4.3 The scramble gate

We realize the contract as an adversarial test (Listing 1, Figure 1). The gate builds a sequence spanning several blocks, evaluates it under the identity table, then reverses the block table (a non-identity permutation that relocates every logical block) and re-evaluates the same tokens. The final logits must be `memcmp`-equal. In the terminology of software testing this is a metamorphic test [5]: the block-table permutation is a metamorphic relation whose expected effect on the output is exactly zero. Because a scrambled table exercises the addressing arithmetic with a mapping that no happy-path

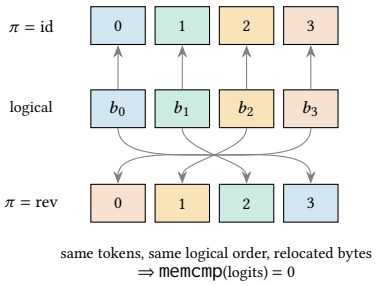

Figure 1: The scramble gate. The same four logical blocks are evaluated under the identity table and under a reversed table that relocates every block; color tracks the logical block. A correct engine produces bitwise-identical logits, and any one-sided addressing bug produces a bitwise difference.

test would produce, the gate tests the contrapositive of Proposition 4.1. Any bitwise difference falsifies one of H1–H3 and localizes a bug that a reference-NMSE comparison could report as 10^{-3} "noise." The gate runs on both the CPU oracle (`--paged-cpu`) and the Metal backend (`--paged-metal`); it is cheap (Section 5) and backend-agnostic.

Listing 1: The placement-invariance gate (paraphrased).

```
eval(identity_session, tokens -> logits_A)
reverse(scrambled_session.block_table)   // non-identity permutation
eval(scrambled_session, tokens -> logits_B)
assert memcmp(logits_A, logits_B) == 0   // bitwise, not tolerance
```

4.4 The gate lattice

The scramble gate is one node in a lattice of gates that every change must pass (Figure 2). Pinned FP32 reference vectors anchor the CPU oracle at exact arg max and NMSE $\leq 5 \times 10^{-5}$ (the oracle's own BF16 KV-storage contract rules out bitwise equality with an FP32 reference; what *is* bitwise is the oracle's stability across engine versions, Section 5). The paged gates anchor both backends' KV addressing bitwise. The cross-backend gates anchor Metal and CUDA to the oracle at exact arg max and NMSE $\leq 10^{-4}$, and the speculation gate requires speculative output to be token-identical to plain greedy decoding. A feature is not merged until the whole lattice is green on real hardware. Re-running the lattice on every engine change is tenable because the gates are fast (Table 2).

5 ORACLE-GATED DETERMINISM

Kipp keeps a scalar CPU forward pass whose BF16 output is byte-identical across the engine's entire evolution: the same logits, bit for bit, as the first release. It is deliberately un-optimized. Its only job is to be a stable oracle. Every other backend is gated against it, with identical arg max at prefill and every checked decode position, and full-logit NMSE at most 10^{-4}. Quantized weight schemes are gated the same way against the BF16 oracle, with exact arg max and a scheme-appropriate NMSE bound.

The kernel-verification literature commonly assumes that optimized GPU kernels are essentially never bitwise-equal to a reference, so one must "model floats as reals" and check within tolerance. Floating-point summation is indeed non-associative, and

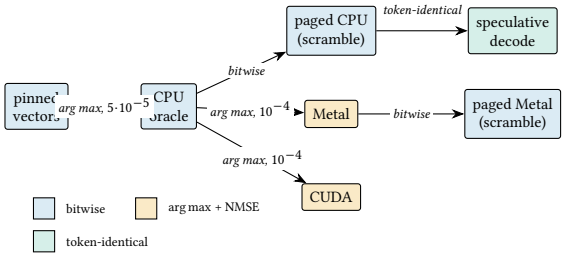

Figure 2: The gate lattice, left to right from anchor to derived gates. Every edge is a machine-checked comparison; node fill encodes the criterion class of its incoming edge. A change merges only when the lattice is green on real hardware. CUDA's node covers its actual scope: contiguous KV, BF16 weights, final-row logits.

Table 2: Gate costs (Qwen3-4B, bf16, single run, idle machine). "bitwise" = `memcmp`-equal full-vocabulary logits. Values from `bench/results/gate-costs.json`.

Gate	Criterion	Cost (s)
`--model` (CPU vs. pinned)	NMSE $\leq 5{\cdot}10^{-5}$, arg max	9.2
`--multilogit`	bitwise	15.5
`--phase2-model` (KV cache)	NMSE $\leq 10^{-6}$, arg max	48.7
`--paged-cpu` (scramble)	bitwise	335.4
`--pooled-cpu` (sharing)	bitwise	1390.2
`--metal-operators`	exact per-kernel	0.6
`--phase3-metal` (Metal vs. CPU)	NMSE $\leq 10^{-4}$, arg max	35.6
`--paged-metal` (scramble)	bitwise	4.1
`--multilogit-metal`	NMSE $\leq 10^{-4}$, arg max	13.5
`--pooled-metal` (sharing)	bitwise + oracle NMSE	184.2

reduction-order differences are a documented source of irreproducibility in both HPC and deep learning [40]. We invert the assumption instead of accepting it: a scalar reference held byte-stable serves as a hard gate, and a real multi-backend engine holds its GPU paths to exact arg max and near machine-precision NMSE against it. The discipline is differential testing [31], applied across backends as CRADLE applies it across deep-learning frameworks [34], with one twist: the reference is not another implementation but a frozen, readable version of this one, which makes every deviation attributable.

Verification is cheap enough to run continuously. Table 2 lists the wall-clock cost of each gate on the evaluation machine. The entire lattice costs about 34 minutes, dominated by the scalar oracle's paged and pooled gates; every GPU-side gate runs in seconds to a few minutes.

6 ENGINE DESIGN AND IMPLEMENTATION

6.1 Architecture

Kipp is small enough to read (Table 3, Figure 3); Objective-C is confined to the Metal bridge and CUDA to its kernels. A single core owns strict GGUF validation against a compiled-in checkpoint registry, the native Qwen tokenizer, sampling, session lifecycle, and

Table 3: Implementation size by component (`wc -l`, July 2026).

Component	Language	Lines
Core: loader, tokenizer, oracle, sessions, block pool, chat, speculation	C11	5,370
Metal backend (bridge + kernels)	ObjC + MSL	3,128
CUDA backend	CUDA C++	1,587
Server	C11	3,017
CLI	C11	694
Tests and gates	C11 + Python	3,602
Benchmark harness	Python	1,346

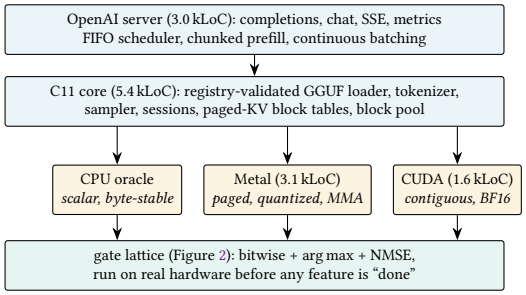

Figure 3: Kipp's architecture. One core drives three backends through a single `eval` contract, and every backend answers to the gate lattice.

the scalar reference forward pass. Backends implement one synchronous `eval` contract (append tokens to a session, produce logits for the last n positions) and own their weight representation, activation scratch, and KV storage. The loader `mmaps` weights from a GGUF-style container and never copies them fully into RAM, so a 4B model serves in about 41 MB of resident memory. No backend falls back to another, and a model outside the registry is rejected rather than accommodated.

6.2 Paged KV manager and block pool

Both the CPU oracle and the Metal backend address KV through the per-sequence block table of Section 4. The physical store is a whole number of 32-position blocks, and the block-table lookup is a two-instruction address computation in the attention read and write kernels; it is the only place placement enters. Cross-request sharing builds on a content-addressed block pool with chained hashes verified by exact token comparison, reference counts, and an LRU free list with revival. Hash collisions can never alias, because every hit is confirmed by `memcmp` over the block's tokens. The sharing discipline is deliberately simple: shared blocks are immutable, complete 32-token blocks, the block being appended to is always private, and a session publishes its blocks to the pool only when it finishes. No copy-on-write is needed, and speculative rollback can never land inside a shared block by construction. Sessions of a pooled model borrow one model-owned physical slab and carry only a block table and token timeline, so adopting a cached prefix is a block-table edit. The whole mechanism is gated bitwise against private-slab sessions on both backends (Section 7.9).

6.3 Kernels

The Metal backend has three matmul families. Single-token decode uses a token-tiled simdgroup matvec with one simdgroup per output row and coalesced loads. Batched prefill uses simdgroup-matrix (MMA) kernels. The BF16 kernel stages activations as BF16 and walks 32-row by 16-token output tiles with FP32 accumulation. The quantized MMA kernels dequantize each 32-weight block into a threadgroup tile once per 16 tokens and run the same fragment loop, reading activations directly in FP32, so the dequantization cost no longer scales with token count and quantized prefill tracks BF16 closely (Section 7.4). Attention comes in two shapes, both in the FlashAttention lineage [7, 8]. Decode uses a split-K streaming online-softmax kernel: eight simdgroups per (head, token) each scan every eighth source position through the block table, and partial softmaxes merge in threadgroup memory; no score matrix is materialized. Prefill uses a tiled simdgroup-matrix flash kernel whose 32-position KV tiles coincide with cache blocks; its arrival also produced the silent-fallback episode of Section 7.13.

The CUDA backend is smaller, and we state its scope exactly: contiguous KV, BF16 weights only, final-row logits only, materialized-score attention, correctness-gated against the oracle on ephemeral cloud GPUs (Table 4). CUDA paging is future work. No Metal code bends to accommodate CUDA or vice versa.

6.4 Quantization

The seven per-layer projections are stored as either Q8_0 (8-bit, 34-byte blocks: fp16 scale, 32 int8 values) or a 4-bit affine scheme (20-byte groups: 16 packed nibbles, fp16 scale, fp16 bias; $w = \text{scale} \cdot q + \text{bias}$) [10, 13, 29]. We call the latter scheme *affine4* (4-bit affine, group size 32) throughout. Embeddings, the LM head, and norms stay BF16. Both schemes are bit-accurate between the CPU and Metal implementations, and quantized artifacts pass the same gate lattice as BF16, including the scramble gates.

6.5 Speculative decoding with adaptive gating

A draft-model-free prompt-lookup matcher [38] proposes the continuation from the longest recent n-gram match. A single multi-logit forward verifies the whole draft, the longest greedy-matching prefix is accepted, and rejected drafts are dropped by truncating the KV back. This is the paged, zero-copy rollback that placement invariance makes safe. Greedy speculative output is token-identical to non-speculative greedy by construction, and a gate asserts it.

On a bandwidth-bound engine, speculation at low acceptance is a net loss (Section 2): ungated prompt-lookup slows grounded decoding to 0.27× (Table 8). Kipp therefore gates drafting adaptively (Algorithm 1). An exponential moving average of per-verify acceptance suspends drafting when it falls below a threshold, and a periodic probe re-enables it when the text becomes repetitive again. Adapting speculation effort to observed acceptance follows SpecDec++ [18], which tunes the draft length online, and BiLD [21], which uses threshold policies between models; Kipp's policy is simpler in that it stops drafting entirely and probes. Gating only chooses between "draft and verify" and "plain decode step." Both emit the greedy sequence, so the token-identity invariant is untouched.

Algorithm 1 Adaptive speculation gating (per decode step)

$ema \leftarrow 0.5$; $active \leftarrow$ **true**; $idle \leftarrow 0$
for each decode step **do**
  if $active$ **or** $idle \geq P$ **then** ▷ P=32 tokens between probes
    draft k tokens; verify; accept prefix a
    $ema \leftarrow \frac{1}{2} ema + \frac{1}{2}(a/k)$; $idle \leftarrow 0$
    if probing **and** $a/k \geq 0.5$ **then**
      $active \leftarrow$ **true**; $ema \leftarrow a/k$
    else if $ema < 0.2$ **then**
      $active \leftarrow$ **false**
    end if
  else
    plain decode step; $idle \leftarrow idle + 1$
  end if
end for

6.6 Serving

A single-threaded `poll()` event loop implements the OpenAI Completions and Chat Completions APIs with continuous batching [46], chunked prefill [1], a native Qwen3 ChatML renderer, the full sampling surface, generated-token log-probabilities, and Prometheus metrics. Multiple choices and concurrent requests decode through one batched eval per step, so each step reads the weights once for all active sequences; the admission cap equals the backend batch limit (32 concurrent generations), so single-choice traffic can fill the whole batch. The scheduler admits requests FIFO with capacity-based backpressure, holds streamed bytes back while they could still grow into a stop string, reaps idle client connections on a deadline, and exports TTFT, queue-wait, and decode-time counters alongside the pool gauges. The same engine also backs a CLI with one-shot generation, perplexity scoring, and a multi-turn chat REPL that re-evaluates only each turn's rendered suffix against the retained KV.

7 EVALUATION

7.1 Setup and methodology

All experiments run on an Apple M5 Max (40-core GPU, 128 GB unified memory) with Qwen3-4B and greedy decoding unless stated, using one discarded warm-up and five measured subprocess runs per configuration. We report medians with median absolute deviation (MAD). The harness (`bench/`) captures separate prefill and decode timers and peak resident memory, and records the engine commit, hardware, and model revision in every result file. Earlier Kipp releases were benchmarked on a base M5 (10-core GPU); all numbers in this paper were re-measured on the M5 Max, and the committed result files self-describe their hardware.

Measurement protocol. Apple-silicon GPU clocks are demand-scaled. Short bursts launched against an idle GPU measure a fraction of sustained throughput — we observed several-fold gaps and large run-to-run swings before controlling for it — so a burst benchmark on a consumer SoC can silently report the ramp, not the engine. Every number in this paper is sustained steady-state: idle machine, multi-minute warm-up, back-to-back single-session measurement, and a MAD-based acceptance bound, as documented

Table 4: Correctness gates (Qwen3-4B). "bitwise" = `memcmp`-equal.

Gate	Result
CPU vs. pinned FP32 reference	arg max exact, NMSE $\leq 5 \cdot 10^{-5}$
CPU vs. its own prior releases	bitwise identical
Metal vs. CPU oracle	arg max exact, NMSE 3×10^{-8}
CUDA vs. CPU oracle (H100)[a]	arg max exact, NMSE $\leq 5.9 \times 10^{-7}$
Paged vs. contiguous (CPU)	bitwise, scrambled table
Paged vs. contiguous (Metal)	bitwise, scrambled table
Spec. vs. greedy decode	token-identical

[a] CUDA scope: contiguous KV, BF16 weights, final-row logits. Revalidated on an ephemeral cloud NVIDIA H100 80GB HBM3 across four checkpoints (`bench/results/cuda-h100-gates.json`).

in `bench/README.md`. The harness also refuses to record Metal results while the bridge reports a kernel-compile fallback, a tripwire earned the hard way (Section 7.13).

For serving experiments we use the standard load metrics [48]: time to first token (TTFT), time per output token (TPOT), and goodput, the fraction of requests meeting both a TTFT and a TPOT service-level objective. For quantization quality we report WikiText-2 perplexity [26], computed over non-overlapping 2,048-token windows with every position after each window's first scored and no burn-in; the protocol is deterministic and identical across schemes, but its absolute values are not comparable to llama.cpp's `perplexity` tool, which discards a per-window burn-in.

7.2 Correctness gates

Table 4 summarizes the gates. The CPU oracle matches the pinned FP32 reference vectors at exact arg max and NMSE $\leq 5 \times 10^{-5}$, and is byte-stable across engine versions (Section 5). Metal matches the oracle at exact arg max and NMSE near 3×10^{-8}, and paged KV is bitwise identical to a contiguous cache under an adversarially scrambled block table on both backends, for all three weight schemes.

7.3 Fault injection: do the gates catch bugs?

The claim of Section 3 is empirical: tolerance-based reference comparisons mask realistic paging bugs that the scramble gate catches. We test it by mutation [9, 20]. A test-only build seeds one deliberate KV-addressing bug at a time [17], and we run both gate families against each mutant. The faults are asymmetric, affecting the read side or the write side only, because Proposition 4.1 itself proves that a mapping error applied identically to both sides is a harmless relocation. The mutation study must model where real bugs live, in one side's copy of the indirection:

(1) *read-block*: attention reads resolve the next logical block;
(2) *read-slot*: attention reads resolve the next slot within the block;
(3) *rollover*: each block's first position is written one past the end of the previous block's physical block, so under the identity table those bytes land exactly where the position belongs;

Table 5: Fault-detection matrix (Qwen3-4B, CPU oracle, 96-token/3-block sequence). "caught" = the gate fails on the mutant. Values from `bench/results/faults.json`.

Seeded fault	NMSE	arg max	Tolerance gate	Scramble gate
(control)	0	same	clean	clean
read-block	6.9×10^{-3}	same	caught	*missed*
read-slot	1.6×10^{-2}	same	caught	*missed*
rollover	0	same	*missed*	**caught**
swap-kv	1.6×10^{0}	flipped	caught	*missed*

(4) *swap-kv*: one layer writes keys into the value store and vice versa.

Table 5 reports, for each mutant, the observed NMSE against the reference, whether the final arg max survived, and each gate's verdict. The detection pattern partitions the faults along the lines of Proposition 4.1.

The placement-commuting faults (read-block, read-slot, swap-kv) corrupt the same logical positions under every block table, so both scramble runs compute identical wrong logits and the bitwise gate passes. The proposition predicts this: the scramble gate cannot see them. Reference testing catches all three, but in two distinct ways. The read-side faults corrupt the logits at NMSE near 10^{-2} *without* flipping the final arg max — only the NMSE bound sees them — so a gate that checked arg max alone, a common lightweight regression test, would have passed every mutant except swap-kv, whose gross corruption (NMSE 1.6×10^{0}, arg max flipped) any check catches.

The rollover fault is the mirror image. Its misaddressed write lands on exactly the right byte under the identity table, so the reference comparison measures NMSE of exactly 0, and no tolerance, however tight, can ever detect it. Only the scramble gate rejects this fault: under a non-identity table the same bug scatters the write into the wrong physical block. The two gate families are therefore complementary rather than redundant. The reference gate covers value corruption, and the scramble gate covers the placement-coupled residue that reference testing cannot reach. That residue is precisely the class of bug that ships silently today, because it only corrupts once a scheduler starts relocating blocks in production.

The study also caught the gate itself. Our original scramble gate spanned two blocks, and the rollover mutant passed it: reversing a 2-entry table is a swap, and "one past the end of the previous block's physical block, wrapping" happens to land on the correct byte again. Reversal over three or more blocks breaks physical adjacency and exposes the fault, so the gate now spans three. Mutation testing evaluated the adequacy of the tests as well as the engine, and improved one of the tests.

7.4 Throughput and quality by weight scheme

Table 6 reports decode and prefill throughput by weight scheme. Decode is bandwidth-bound, so quantization scales it as it shrinks the weight bytes streamed per token. Prefill is compute-bound, and the quantized MMA kernels hold it near BF16 (488 and 509 vs. 527 tok/s, within 4–8%): dequantizing each 32-weight block once

Table 6: Throughput (tok/s), Qwen3-4B, Apple M5 Max Metal, median of 5. Median absolute deviations are ≤1 tok/s throughout (per-run distributions in the committed result files).

Weight scheme	Decode	Prefill
BF16	60.7	527
Q8_0	97.9	488
affine4	130	509

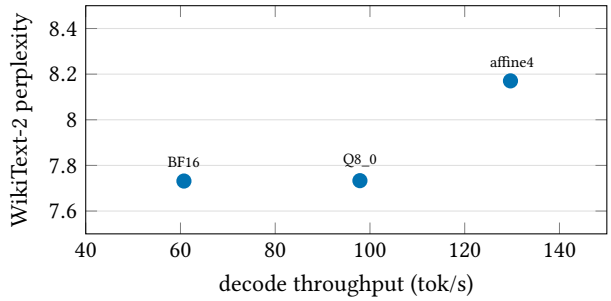

Figure 4: Quality–speed tradeoff by weight scheme (Qwen3-4B, M5 Max Metal): WikiText-2 perplexity against decode throughput, one point per scheme. Down and to the right is better.

per 16-token tile keeps the added arithmetic off the critical path, so quantization buys its decode speedup without surrendering prefill.

Speed is only half the tradeoff. Figure 4 plots each scheme's WikiText-2 perplexity against its decode throughput, which is the plot a deployment decision actually needs: how much quality each streamed byte buys. Q8_0 is quality-free at this precision of measurement (7.733 vs. 7.731 perplexity, +0.022%) while lifting decode from 60.7 to 97.9 tok/s; affine4 pays +5.7% perplexity (8.171) for the fastest decode of the three (130 tok/s).

7.5 Model-size scaling

Decode throughput on bandwidth-bound hardware should scale inversely with the bytes streamed per token, independent of which checkpoint streams them. Figure 5 checks this across the family (0.6B, 4B, 8B), so the 4B results of Section 7.4 can be read as representative rather than cherry-picked: BF16 decode falls from 269 through 60.7 to 33.5 tok/s as parameters grow 0.6 → 4 → 8B, and Q8_0 halves the streamed bytes at 8B for 57.8 tok/s.

7.6 Context scaling

Figure 6 reports throughput against prompt length (Q8_0, 32-token decode, median of 3; `bench/results/ctx-*.json`). Decode degrades gracefully, from 98.4 to 26.0 tok/s between a 3- and a 12,800-token context, as the per-step KV read grows against the fixed weight stream. Prefill peaks near 485 tok/s at an 800-token prompt and then declines steadily with context, the expected shape once attention's quadratic term grows against the linear matmul work; at 12,800 tokens prefill still sustains over a third of its peak.

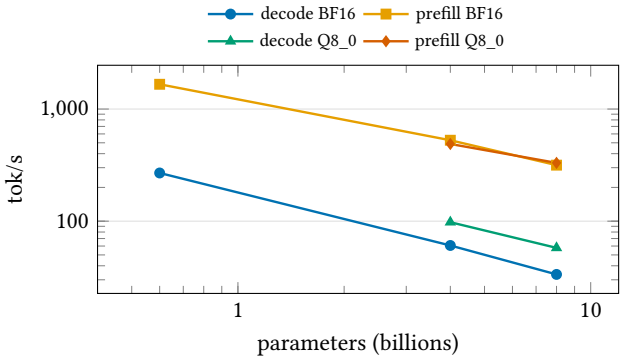

Figure 5: Throughput across the Qwen3 dense family (M5 Max Metal). Decode tracks streamed weight bytes per token; prefill tracks compute.

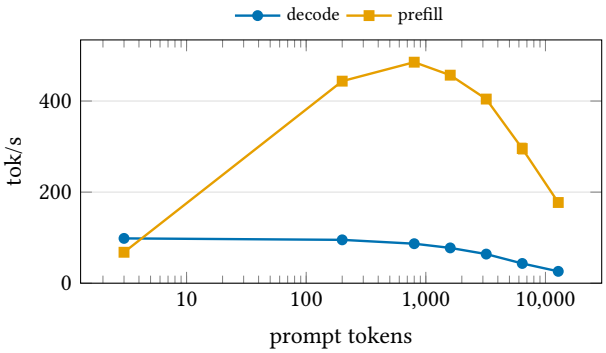

Figure 6: Context scaling (Qwen3-4B Q8_0, M5 Max Metal; error bars show MAD). Decode falls as the per-step KV read grows; prefill peaks at short context and declines as attention's quadratic cost grows against the linear matmul work.

7.7 Batch scaling

Because decode is bandwidth-bound, aggregate server throughput scales with the number of concurrently decoding sequences: each batched step reads the weights once for all choices. Figure 7 plots aggregate throughput on the pooled Q8_0 server for one request with n parallel choices and for n independent concurrent connections, against the ideal of n times the single-stream rate (sampled decoding, median of 3; `bench/results/server-batch.json`). One request with n=1/2/4/8 choices delivers 76.9, 96.8, 89.8, and 80.1 tok/s aggregate; 1/2/4/8 independent connections deliver 55.9, 61.3, 78.5, and 80.8. The two series differ at n=1 because they measure different aggregates: parallel choices share one prompt's prefill across the batch, while concurrent connections each pay their own prefill inside the measured window (the ideal-linear reference is therefore anchored on the choices series). The shapes then diverge instructively. Independent connections scale monotonically through n=8, as batched decode amortizes the weight stream across every admitted request. Parallel choices peak at n=2 and then *decline*: each additional choice adds per-step CPU-side sampling work in the single-threaded scheduler (these runs sample at temperature 0.8,

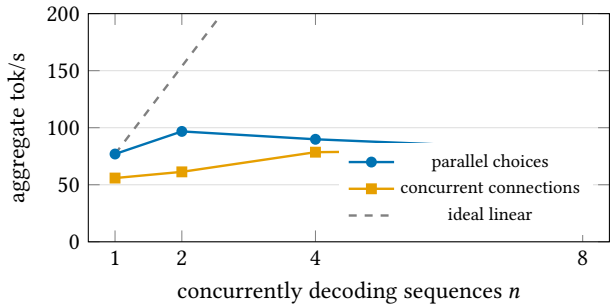

Figure 7: Batch scaling on the pooled Q8_0 server (M5 Max). The dashed line is ideal linear scaling from the single-stream rate. Independent connections scale monotonically; parallel choices peak at n=2, where per-step CPU-side sampling begins stalling the GPU long enough for demand-scaled clocks to sag.

so the greedy fast path does not apply), the GPU idles longer between batched steps, and the demand-scaled clocks of Section 7.1 sag in response — a serving-on-consumer-SoC effect a fixed-clock datacenter GPU would not show. The gap to the ideal line is that scheduling cost made visible.

7.8 Serving under load

Batch scaling measures a closed system; serving is an open one. We drive the server with open-loop Poisson arrivals of streamed completion requests at increasing offered rates and record per-request TTFT and TPOT percentiles and goodput under a fixed SLO (Figure 8; `bench/results/load.json`). This is the view a capacity planner needs: the offered rate at which tail latency leaves the knee, and how much of the offered load still meets its SLO beyond it.

The verdict for this workload (32-request bursts of 348-token prompts): at 0.5 req/s the server holds its service-level objective for the large majority of requests (median TTFT 1290 ms), and goodput collapses to 0.00 from 2 req/s on. Past the knee the overload lands where an admission-controlled FIFO puts it: p99 TTFT grows monotonically with queue depth to 16.6 s at 4 req/s, aggregate output tops out near 61.9 tok/s, and per-token latency rises only with batch occupancy — TPOT roughly triples from the lowest to the highest offered rate as more requests share each batched decode step, then flattens (Figure 8, right). The system behaves exactly as the batch-scaling analysis predicts — admission control and the FIFO queue protect correctness, decode throughput degrades only by batch sharing, and time to first token absorbs the overload.

7.9 Cross-request prefix sharing

Prefix sharing is live in the serving path. A finished session publishes its full 32-token blocks to the content-addressed pool, and a new request adopts the longest published prefix at admission, re-evaluating only the partial tail. On a 6,890-token prompt sent twice (`bench/results/prefix-reuse.json`), the second request adopts 6,880 tokens and its prefill time collapses from 30.4 s to 174 ms, a 175× reduction in time to first token, with byte-identical output.

Table 7: Pooled-sharing gate scenarios (`--pooled-cpu`, `--pooled-metal`), each run on CPU and Metal for BF16, Q8_0, and affine4 weights.

Scenario	Criterion
Serial prefix sharing	bitwise vs. private-slab session
Mixed shared/private batch	bitwise vs. private-slab sessions
Pool exhaustion	clean failure; admission delays, never corrupts
Truncation (spec. rollback)	bitwise; never enters a shared block
Eviction and revival	bitwise after LRU eviction

The correctness side is gated rather than assumed: Table 7 lists the pooled-gate scenarios, each proven on both backends for BF16 and quantized weights. This is the payoff of Section 4: block adoption is a block-table edit, and the scramble gate is what makes editing tables a safe operation.

7.10 Speculative decoding

Table 8 and Figure 9 report acceptance rate α, block efficiency (accepted tokens per target forward), and wall-clock speedup across four workloads, with and without adaptive gating. The A/B is paired: for each workload the harness measures plain decode, ungated speculation, and gated speculation adjacently within one session, so both speedup columns share a single baseline (the gap between the two mechanically identical repetitive configurations, 2.10× vs. 2.27× at duty 1.0, bounds residual run-to-run variance). One honest denominator note: a greedy-sampling fast path landed in the same revision as this measurement session, so the plain-decode baselines are faster than in earlier drafts and every speculation ratio is correspondingly compressed — same absolute wall-clock wins, smaller multipliers. Ungated prompt-lookup lifts copy-heavy decode from 90.6 to 190 tok/s (2.10×) at α=1.0. However, it slows code to 0.73× and grounded text to as little as 0.27×: the k+1-token verify does not amortize weight streaming at small draft batch, so each speculative step costs close to k+1 decodes while advancing by only accepted+1 tokens. The adaptive gate suspends drafting on exactly those workloads — its duty cycle (the fraction of steps that draft) makes the mechanism visible — and holds every workload at or above 0.84×, pushing code past parity (1.24×) while keeping the copy-heavy win (2.27×). In every configuration the emitted tokens are identical to greedy decoding; speculation changes only the wall clock.

7.11 Comparison with llama.cpp

We compare against llama.cpp [13] on identical hardware and the same pinned Qwen3-4B revision, converted independently with llama.cpp's own tooling, at the pinned llama.cpp commit recorded with the exact commands in `bench/results/llamacpp-qwen3-4b.json` (`llama-bench`, 2,048-token prefill, 256-token decode, five repetitions). Table 9 pairs llama.cpp's numbers with Kipp's decode benchmark and Kipp's matched 2,048-token prefill. The result cuts both ways. On decode, the phase that dominates serving cost, Kipp is faster in every matched scheme, about 1.73× at 8-bit (97.9 vs. 56.5 tok/s),

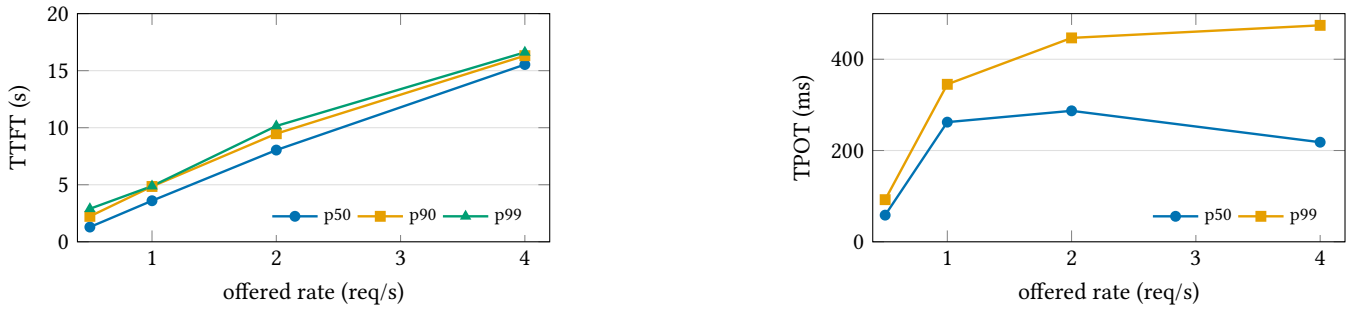

Figure 8: Serving under open-loop Poisson load (Qwen3-4B Q8_0, pooled server, M5 Max): TTFT and TPOT percentiles against offered request rate. TTFT grows with queue depth; TPOT rises with batch occupancy and then flattens — the overloaded server queues and shares each decode step, it does not thrash.

Table 8: Prompt-lookup speculation by workload (Qwen3-4B Q8_0, M5 Max, 256-token decode), controlled A/B, median of 3. "Gated" adds the adaptive gate of Algorithm 1; "duty" is its drafting duty cycle. Speedup <1 is slower than plain decode.

Workload	α	Blk. eff.	Ungated	Gated	Duty
Repetitive	1.00	4.00	2.10×	2.27×	1.00
Code	0.19	1.09	0.73×	1.24×	0.06
Grounded	0.00	1.00	0.27×	0.90×	0.04
Open-ended	0.09	1.07	0.62×	0.84×	0.10

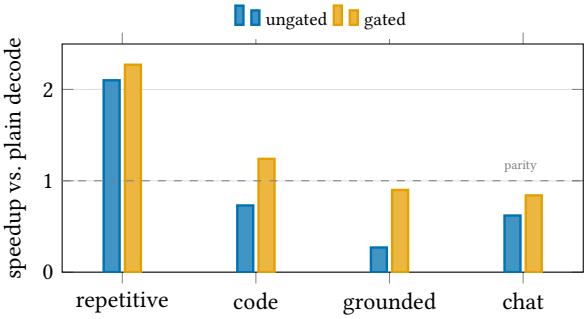

Figure 9: Speculation speedup by workload, with and without the adaptive gate. The dashed line is parity with plain decode: gating preserves the repetitive-workload win, pushes code past parity, and pulls the remaining workloads back to it.

Table 9: llama.cpp head-to-head (Qwen3-4B, M5 Max Metal; tok/s). Decode: Kipp median of 5 (348-token prompt, 64-token decode) vs. `llama-bench` mean of 5 (n=256). Prefill: matched 2,048-token prompt on both. llama.cpp Q4_0 is compared against Kipp affine4 (schemes differ; see text).

	Decode		Prefill (2k)	
Weights	Kipp	llama.cpp	Kipp	llama.cpp
BF16	**60.7**	35.0	481	**2174**
Q8_0	**97.9**	56.5	441	**2329**
affine4 / Q4_0	**130**	89.1	466	**2420**

and its affine4 outruns llama.cpp's Q4_0 (130 vs. 89.1). The verification discipline costs nothing where it matters most. On prefill llama.cpp leads, about 4.5× at BF16 (2174 vs. 481 tok/s at 2,048 tokens) and somewhat more at the quantized schemes, reflecting years of kernel engineering (larger tiles, threadgroup-level pipelining, autotuned dispatch) that Kipp's readable single-kernel-per-op design does not attempt. Kipp is not the fastest prefill engine and does not try to be; its scope is one pinned family with no autotuning, and we present this number as the measured price of a readable, verifiable prefill path today. Three caveats on fairness. Both engines were measured in the same idle steady-state session (Section 7.1), but `llama-bench` reports mean±stddev while Kipp reports median±MAD. The decode configurations differ (64- vs. 256-token generations), but batch-one decode rate is insensitive at these depths — our context sweep moves only from 98.4 to 95.2 tok/s between 3 and 200 tokens of context — so both measure the same steady rate. And the 4-bit schemes differ (llama.cpp Q4_0 is scale-only; affine4 carries a bias), so that column is close cousins rather than twins.

7.12 Memory

Peak process RSS is about 41 MB for the 4B model regardless of quantization, since the loader `mmap`s weights and pages them in on demand. Note that peak RSS understates true unified-memory residency for GPU-touched `mmap` pages, so we report artifact size alongside RSS and never present RSS as the total memory needed to run the model.

7.13 Ablations

A silent fallback defeats every correctness gate. The sharpest lesson in this paper was self-inflicted, and we report it in full. The tiled flash-prefill kernel of Section 6.3 landed with a loop variable named `fragment` — a reserved identifier in the Metal Shading Language. The runtime compile of the *entire* simdgroup-matrix library therefore failed with a syntax error, and the bridge silently fell back

to vector kernels for every matrix matmul *and* for the new kernel itself, which as a result had never actually executed: its gates, and an entire benchmark campaign, exercised the fallback. The fallback is a correct implementation, so every correctness gate in the lattice passed throughout — while prefill shipped 35–75% slower than the engine's real capability, quantized prefill appeared to trail BF16 at exactly the superseded vector path's throughput, and context-scaling appeared to collapse. The only symptom was one warning line on `stderr`, which every harness discarded. A one-identifier rename restored everything: BF16 prefill at a 348-token prompt rose from 355 to 527 tok/s, Q8_0 from 218 to 488 (restoring the quantized-MMA near-parity the design had always provided), and Q8_0 at 2,048 tokens from 70 to 441. The moral sharpens this paper's thesis rather than softening it: correctness gating is necessary but not sufficient. A silent within-engine fallback passes every value-level check by construction, so fallbacks must be loud, and performance needs its own tripwire — the harness now refuses to record Metal results while the fallback warning is present. Every kernel change in this line was, and remains, admitted through the same lattice as any other feature: exact arg max, per-scheme NMSE, bitwise scramble.

Verify-path reduction order. The MMA work also produced a cautionary result. Routing speculative verify rounds through the matrix kernels broke the spec-equals-greedy gate, because the different summation order flipped near-tie arg maxes. Verify rounds now force the vector kernels, whose per-token reduction order is bitwise-identical to decode's, and the gate is green again. The contract caught its own optimization.

Gate sensitivity. The mutation study of Section 7.3 doubles as an ablation of the gate lattice. Removing the scramble gate leaves the rollover fault undetectable in principle. Removing the NMSE bound and keeping only arg max would pass three of the four mutants. Shrinking the scramble sequence back to two blocks re-masks the rollover fault. Every component of the lattice has at least one mutant that only it catches.

Adaptive gating. The paired columns of Table 8 isolate the gate's effect: it preserves the repetitive-workload win, pushes code past parity, and converts the remaining low-acceptance losses to near-parity.

8 DISCUSSION AND LIMITATIONS

What the contract does not cover. Placement invariance is claimed only for retained KV, not for logical reordering or prefill-column permutation (Section 4). The scramble gate exercises one adversarial permutation (reversal) per run. It localizes one-sided addressing bugs, but a bug that only manifests under a specific table shape could in principle require other permutations. These are cheap to add, since any non-identity permutation is a valid gate.

Porting the discipline, and future work. Nothing in the scramble gate is specific to Kipp. Any paged engine with a per-sequence block table can evaluate a multi-block prompt twice, once under identity and once under a reversal, and `memcmp` the logits, provided its kernels satisfy H3 within a run (fixed reduction order for a fixed batch shape), which batch-invariant kernels [16] already provide. The silent-fallback episode of Section 7.13 argues for a companion discipline: any within-engine fallback path should fail loudly or be recorded in measurement provenance, because value-level gates cannot see it. We would welcome the gate's adoption in vLLM or SGLang; it is a few dozen lines against either engine's test harness. The most promising extension is tree-shaped speculative rollback [3, 27, 28]: chain methods roll back by cheap truncation and dense-buffer tree methods pay a gather-copy, so reclaiming rejected tree branches by block-table unlinking, under a bitwise-safety gate, is exactly the kind of operation placement invariance makes trustworthy.

8.1 Limitations

- *Scope.* Kipp is a single-node, single-GPU engine for one pinned model family; distributed serving and heterogeneous placement are out of scope.
- *CUDA.* The CUDA backend is contiguous, BF16-only, final-row logits, gated on ephemeral cloud GPUs (most recently a NVIDIA H100 80GB HBM3 across four checkpoints, Table 4). Extending paging and the scramble gate to CUDA is mechanical (the kernels mirror Metal's structure) but unverified, so we do not claim it.
- *One permutation per run.* The scramble gate reverses the table; other adversarial permutations are possible and cheap but not yet exercised.
- *Quality metric.* Quantization quality is evaluated by perplexity only; task-suite accuracy [26] is future work.
- *Oracle scope.* The oracle pins greedy logits; it does not characterize sampling distributions beyond the logits it checks.

9 RELATED WORK

PagedAttention [22] introduced the block table, and SGLang's RadixAttention [47] shares prefixes across calls via a radix tree. A growing family of systems moves KV around at runtime: Llumnix migrates running requests and their caches between instances [42], InfiniGen spills KV between GPU and host memory [23], Mooncake ships it between prefill and decode clusters [36], CachedAttention persists it across conversation turns [12], and Prompt Cache reuses precomputed attention states across prompts [14]. vAttention revisits the layout question entirely, backing contiguous virtual KV with demand paging [35], and production engines such as TensorRT-LLM [33] and TGI [19] relocate blocks internally. All of these systems evaluate throughput and cache-hit rate; none tests placement invariance as a named property (Table 1). CacheGen [30] makes an interesting contrast, since it compresses KV lossily in transit and deliberately gives up the bitwise equality that our gate demands. On-device engines such as llama.cpp [13] and MLX [15] target the same hardware class as Kipp's Metal backend.

He and Thinking Machines Lab [16] trace run-to-run nondeterminism to batch-size-dependent kernel reductions and restore bitwise determinism with batch-invariant kernels. That work fixes the reduction-order axis; our contract fixes the orthogonal placement axis and adds a byte-stable scalar oracle as a hard cross-backend gate. On the testing side, the scramble gate is a metamorphic test

[5], and the oracle discipline is differential testing [31] in the lineage of CRADLE's cross-framework validation [34]. Symbolic checkers such as GKLEE [25] verify GPU kernels exhaustively but do not scale to a full transformer forward pass, and fuzzers for deep-learning libraries [43] find crashes rather than wrong-computation bugs, which require an oracle. Our fault study applies classical mutation testing [9, 20] and software fault injection [17] to an inference engine's memory system. MLPerf [37] standardizes performance reporting; we extend the same rigor to correctness claims.

FlashAttention [6, 7, 39] made exact attention IO-efficient; FlashDecoding [8] specializes the split-K streaming shape for decode, and FlashInfer [45] generalizes paged and block-sparse attention kernels behind a serving-oriented interface. Kipp's streaming online-softmax Metal kernel is in this lineage. Post-training quantization [10, 29] and GGUF k-quants [13] inform Kipp's block-quantized weights, and Li et al. [26] systematize quality evaluation for quantized LLMs. Speculative decoding [4, 24] accelerates decode, with draft-model-free [11, 38], self-drafting [3, 27, 28], and adaptive [18, 21] variants. Kipp's contribution here is the acceptance-gated policy and the token-identity gate, not a new drafting scheme. DistServe [48] defines the goodput-oriented serving metrics our load evaluation adopts.

10 CONCLUSION

Kipp shows that the placement invariance every paged inference system relies on can be a tested contract rather than an assumption, and that the stakes are measurable by mutation. A realistic block-rollover bug is bitwise invisible to every identity-mapped reference test, yet a scramble gate that costs seconds on GPU catches it immediately, while read-side corruption sails past arg max checks and falls only to a tight NMSE bound. The gates are complementary, each with mutants that only it catches, and the mutation study repaired a blind spot in the gate itself. A byte-stable scalar oracle holds real GPU backends to exact arg max, and every feature in the engine, from quantization and MMA prefill to batching, adaptive speculation, and pooled KV sharing, landed through the same lattice. The result is a small, continuously verifiable engine whose correctness, and every benchmark number in this paper, is reproducible by re-execution — and a discipline that any paged engine can adopt for the cost of a reversed block table and a `memcmp`.

ARTIFACT AVAILABILITY

Kipp is open source under the MIT license:
https://github.com/Polished-Snow/Kipp
This paper was built from commit `ba1a3c0` of that repository (branch `paper/verifiable-paged-attention`). An archived artifact with a DOI will accompany the final version. `REPRODUCE.md` in the repository documents the full build, gate, and benchmark reproduction path, and Appendix A maps each claim in this paper to the command that checks it. Every measured number is generated into the paper from a committed `bench/results/*.json` by `make paper-data` and verified by `make paper-check`.

REFERENCES

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. https://arxiv.org/abs/2403.02310

[2] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2305.13245

[3] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. 2024. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2401.10774

[4] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318* (2023). https://arxiv.org/abs/2302.01318

[5] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic Testing: A Review of Challenges and Opportunities. *Comput. Surveys* 51, 1 (2018), 1–27.

[6] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2307.08691

[7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*. https://arxiv.org/abs/2205.14135

[8] Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash-Decoding for Long-Context Inference. Stanford CRFM / PyTorch blog. https://crfm.stanford.edu/2023/10/12/flashdecoding.html.

[9] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. 1978. Hints on Test Data Selection: Help for the Practicing Programmer. *IEEE Computer* 11, 4 (1978), 34–41.

[10] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.17323

[11] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. 2024. Break the Sequential Dependency of LLM Inference Using Lookahead Decoding. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2402.02057

[12] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Cost-Efficient Large Language Model Serving for Multi-turn Conversations with CachedAttention. In *USENIX Annual Technical Conference (ATC)*. https://arxiv.org/abs/2403.19708

[13] Georgi Gerganov and llama.cpp contributors. 2023. llama.cpp: LLM Inference in C/C++ (GGUF format and k-quants). https://github.com/ggml-org/llama.cpp. GGUF container and k-quant mixed-bit quantization; no associated paper.

[14] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems (MLSys)*. https://arxiv.org/abs/2311.04934

[15] Awni Hannun, Jagrit Digani, Angelos Katharopoulos, and Ronan Collobert. 2023. MLX: Efficient and Flexible Machine Learning on Apple Silicon. https://github.com/ml-explore/mlx.

[16] Horace He and Thinking Machines Lab. 2025. Defeating Nondeterminism in LLM Inference. Thinking Machines Lab: Connectionism (blog). https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/.

[17] Mei-Chen Hsueh, Timothy K. Tsai, and Ravishankar K. Iyer. 1997. Fault Injection Techniques and Tools. *IEEE Computer* 30, 4 (1997), 75–82.

[18] Kaixuan Huang, Xudong Guo, and Mengdi Wang. 2025. SpecDec++: Boosting Speculative Decoding via Adaptive Candidate Lengths. In *Conference on Language Modeling (COLM)*. https://arxiv.org/abs/2405.19715

[19] Hugging Face. 2023. Text Generation Inference. https://github.com/huggingface/text-generation-inference.

[20] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (2011), 649–678.

[21] Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W. Mahoney, Amir Gholami, and Kurt Keutzer. 2023. Speculative Decoding with Big Little Decoder. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*. https://arxiv.org/abs/2302.07863

[22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. https://arxiv.org/abs/2309.06180

[23] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient Generative Inference of Large Language Models with Dynamic KV Cache

Management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. https://arxiv.org/abs/2406.19707
[24] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2211.17192
[25] Guodong Li, Peng Li, Geof Sawaya, Ganesh Gopalakrishnan, Indradeep Ghosh, and Sreeranga P. Rajan. 2012. GKLEE: Concolic Verification and Test Generation for GPUs. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 215–224.
[26] Shiyao Li, Xuefei Ning, Luning Wang, Tengxuan Liu, Xiangsheng Shi, Shengen Yan, Guohao Dai, Huazhong Yang, and Yu Wang. 2024. Evaluating Quantized Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2402.18158
[27] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2406.16858
[28] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. https://arxiv.org/abs/2401.15077
[29] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. In *Proceedings of Machine Learning and Systems (MLSys)*. https://arxiv.org/abs/2306.00978
[30] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. https://arxiv.org/abs/2310.07240
[31] William M. McKeeman. 1998. Differential Testing for Software. *Digital Technical Journal* 10, 1 (1998), 100–107.
[32] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. 2025. Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems. *Comput. Surveys* (2025). https://arxiv.org/abs/2312.15234
[33] NVIDIA Corporation. 2023. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM.
[34] Hung Viet Pham, Thibaud Lutellier, Weizhen Qi, and Lin Tan. 2019. CRADLE: Cross-Backend Validation to Detect and Localize Bugs in Deep Learning Libraries. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*.
[35] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. https://arxiv.org/abs/2405.04437
[36] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. In *23rd USENIX Conference on File and Storage Technologies (FAST)*. https://arxiv.org/abs/2407.00079
[37] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, et al. 2020. MLPerf Inference Benchmark. In *Proceedings of the 47th Annual International Symposium on Computer Architecture (ISCA)*. https://arxiv.org/abs/1911.02549
[38] Apoorv Saxena. 2023. Prompt Lookup Decoding. https://github.com/apoorvumang/prompt-lookup-decoding. n-gram-based draft-model-free speculative decoding.
[39] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. 2024. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. https://arxiv.org/abs/2407.08608
[40] Sanjif Shanmugavelu, Mathieu Taillefumier, Christopher Culver, Oscar Hernandez, Mark Coletti, and Ada Sedova. 2024. Impacts of Floating-Point Non-Associativity on Reproducibility for HPC and Deep Learning Applications. arXiv preprint arXiv:2408.05148. https://arxiv.org/abs/2408.05148
[41] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced Transformer with Rotary Position Embedding. *Neurocomputing* 568 (2024). https://arxiv.org/abs/2104.09864
[42] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. https://arxiv.org/abs/2406.03243
[43] Anjiang Wei, Yinlin Deng, Chenyuan Yang, and Lingming Zhang. 2022. Free Lunch for Testing: Fuzzing Deep-Learning Libraries from Open Source. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. https://arxiv.org/abs/2201.06589
[44] An Yang and Qwen Team. 2025. Qwen3 Technical Report. arXiv preprint arXiv:2505.09388. https://arxiv.org/abs/2505.09388
[45] Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. 2025. FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving. In *Proceedings of Machine Learning and Systems (MLSys)*. https://arxiv.org/abs/2501.01005
[46] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 521–538. https://www.usenix.org/conference/osdi22/presentation/yu
[47] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. https://arxiv.org/abs/2312.07104
[48] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. https://arxiv.org/abs/2401.09670

A ARTIFACT: CLAIMS TO COMMANDS

Every correctness claim maps to a gate command, and every number maps to a committed result file. Build and gate commands (see also `REPRODUCE.md`):

```
make cpu metal server-metal      # binaries
make test test-sanitize          # hermetic units + ASan/UBSan
# model gates (direct; $M = GGUF, $V = pinned vectors):
build/kipp_test        --model         $M $V   # argmax + NMSE<=5e-5
build/kipp_test        --paged-cpu     $M $V   # scramble, bitwise
build/kipp_test        --multilogit    $M $V
build/kipp_test        --phase2-model  $M $V
build/kipp_test_metal  --metal-operators
build/kipp_test_metal  --phase3-metal  $M $V   # argmax + 1e-4
build/kipp_test_metal  --paged-metal   $M $V   # scramble, bitwise
build/kipp_test_metal  --multilogit-metal $M $V
make test-server                 # OpenAI API surface
tools/ops/verda_cuda_gate.sh     # CUDA gates, ephemeral cloud GPU
# mutation study (test-only build):
make build/kipp_test_fault
KIPP_FAULT=<1..4> build/kipp_test_fault --phase2-model $M $V
KIPP_FAULT=<1..4> build/kipp_test_fault --paged-cpu    $M $V
# benchmarks (each writes a self-describing bench/results/*.json):
python3 tools/bench.py ...             # decode/prefill by scheme
      and size
python3 bench/spec_bench.py --gate off # ungated speculation
      baseline
python3 bench/spec_bench.py --gate on  # adaptive-gated speculation
python3 bench/server_bench.py ...      # batch scaling
python3 bench/load_bench.py ...        # open-loop serving load
python3 bench/prefix_bench.py ...      # cross-request prefix reuse
python3 bench/ppl_bench.py ...         # WikiText-2 perplexity by
      scheme
# paper numbers (regenerate + verify the macro/data bindings):
make paper-data paper-check
```